

Security Audit Skill — Hardening & Robustness Suggestions

Where the current build is weak, and what would make it materially stronger. Ordered by value per unit of effort, with the honest limitations stated first, because several suggestions only make sense once you know what the skill genuinely cannot do today.

0. Known limitations of the current build

These are properties of the design, not bugs. Any hardening plan should start here.

Limitation	Consequence
Regex-based sensing, not AST or dataflow	The deterministic layer cannot follow a tainted value across function boundaries. Reachability is established by the model reading code, which is good but not exhaustive, and it scales with attention rather than with certainty.
No execution	Nothing is confirmed by exploitation. Every finding is a static inference.
Findings are not reproducible run to run	The rule pass is deterministic; triage is not. Two runs on identical code can differ in wording, in <code>Possible</code> -tier findings, and slightly in score.
Dependency advisories usually unverified	Without an MCP server or network access, dependency status is <i>unknown</i> . The report says so, but a reader who skims will miss it.
Coverage is unmeasured	The report says what was checked; it cannot say what fraction of the attack surface that represents.
No cross-file dataflow	A sink in one file fed by a source in another is found only if the model happens to connect them.
Cost measurement is transcript-dependent	Works in Claude Code; a different harness would need a different collector.
Legacy-origin classification is inferred	Without auditing the legacy source too, <code>legacy-inherited</code> vs <code>migration-introduced</code> is a judgment call, not evidence.

1. High value, low effort

1.1 Golden-corpus regression tests — **DELIVERED**

Shipped as `tests/run_corpus.py` + `tests/corpus/`. 109 pattern-compile checks and 14 fixture cases covering .NET, Python, JavaScript and the universal pack.

It paid for itself on the first run, finding three defects that had survived manual review — most seriously, `UNI-INJ-01` was inverted: it missed concatenated SQL and flagged safe parameterized interpolation instead. See the design document for the full account.

Remaining work on this item: extend the corpus to Java, PHP/Go/Ruby, SQL/legacy and config/IaC, which currently have compile coverage but no fixtures. The rule is that **every new rule arrives with at least one reject case** drawn from its own `falsePositiveHints`.

The original rationale, still true:

Build a `tests/corpus/` of small, deliberately vulnerable and deliberately clean files per stack — each with an expected-findings manifest:

```
tests/corpus/dotnet/sqlite-concat.cs    → expects DN-INJ-01, severity High
tests/corpus/dotnet/sqlite-parameterized.cs → expects NOTHING (the false-positive guard)
tests/corpus/dotnet/authorize-present.cs → DN-AUTHZ-01 must NOT fire
```

Then a runner that greps each rule against the corpus and asserts the expected set. The false-positive cases matter more than the true-positive ones: precision is what makes the report credible, and precision regressions are invisible without a test that asserts silence.

Extend the pattern-compile check already used in this build (`printf 'x' | rg -e "$pat"`, `exit ≥ 2` = broken) into that runner so both properties are enforced in CI.

Effort: a day for the harness plus ~4 files per pack. **Payoff:** rule packs become safely editable, which is the difference between a living tool and a frozen one.

1.2 Triage consistency harness

Run the same target three times and diff the findings sets. Report which findings are stable, which appear intermittently, and how much the score moves.

This does not make triage deterministic — it makes its variance *visible*, which is what you need before anyone puts a score in a release gate. If the same codebase scores 71 and 78 on consecutive runs, that is a fact the users of the score must know.

Effort: half a day. **Payoff:** you learn whether the score is gate-quality or indicative-only.

1.3 Rule metadata the report can use

Add to each rule: `confidencePrior` (how often this pattern is a real finding), `lastReviewed`, and `references`. Then the report can note *"this rule historically has a high false-positive rate"* next to a `Possible` finding, and stale rules become visible instead of quietly rotting.

1.4 Machine-readable output for pipelines

Emit **SARIF 2.1.0** alongside the HTML. It is the standard static-analysis interchange format, and it is what unlocks integrations you would otherwise build by hand: GitHub code scanning annotations on the PR diff, Azure DevOps and GitLab security tabs, and Defect Dojo / SonarQube ingestion.

`findings.json` already carries everything SARIF needs; this is a mapping, not new analysis.

Effort: a day. **Payoff:** findings appear inline on the pull request, which is where they actually get fixed.

1.5 Suppression file with expiry

A `.security-audit-ignore` at the target root:

```
- rule: UNI-CRY-01
  path: src/Cache/KeyBuilder.cs
  reason: "MD5 used as a cache key hash, not a security control"
  reviewedBy: gsegura
  expires: 2027-03-01
```

Two properties that make this safe rather than a loophole: suppressions are **reported** in a dedicated report section rather than being invisible, and they **expire**, so an accepted risk is re-examined instead of becoming permanent by neglect. An unexpiring suppression file is how static analysis programs die.

1.6 Make the "unverified dependencies" caveat unmissable

Today it is a Limitations bullet. Promote it: when dependencies were not verified, render the dependency category tile with an explicit "not assessed" state in the dashboard itself, not only in the table. The most dangerous failure mode of this report is a reader inferring "clean" from "quiet".

2. High value, medium effort

2.1 Real taint analysis for the top three sink classes

Regex finds sinks; it cannot prove a source reaches one. For SQL injection, command injection and XSS — the three where reachability decides everything — a lightweight source→sink pass would convert a large fraction of **Likely** findings into **Confirmed** ones and eliminate most false positives.

Options, cheapest first:

1. **Model-driven, structured.** Have the skill enumerate sources and sinks, then explicitly trace each pair and record the path in the finding. No new dependency; it formalizes what triage already does and makes the reasoning auditable.
2. **Language-native tooling where present, and only when the user approves it.** Roslyn analyzers for .NET, `semgrep --config auto` if installed, `bandit`, `gosec`, `brakeman`. These are real AST/dataflow engines. They break the zero-dependency property, so they must stay strictly opt-in — detect, ask, use if approved, and record in the report which engine ran.
3. **Use Python's own `ast` module for Python targets.** Now cheap: `bin/ensure_python.*` already resolves an interpreter when one exists, and `ast` is stdlib — real dataflow for Python codebases with no new dependency. This is the natural first taint implementation, because the analysis engine and the audited language coincide.
4. **Bundle a parser.** Tree-sitter grammars would give real AST queries for every language, but they need native wheels — a genuine install, not just an interpreter. Only worth it if the zero-install constraint is relaxed.

Recommendation: do (1) now, add (3) for Python targets, offer (2) as opt-in, and treat (4) as a fork for teams that want depth over portability.

2.2 Audit both sides of a migration

For Modernizer work this is the highest-value change available. Add `--legacy-path <dir>` and audit the source system alongside the generated output. That turns `origin` from inference into evidence and separates three very different things:

Class	Meaning	Who acts
<code>legacy-inherited</code>	Present in both	The business — pre-existing risk, now possibly more exposed
<code>migration-introduced</code>	Absent in source, present in output	The Modernizer team — fix the generator, not the output
<code>migration-exposed</code>	Code unchanged, but the environmental control that protected it is gone	Architecture — this is the most-missed class

`migration-exposed` deserves special emphasis in any hardening plan. Legacy systems lean heavily on controls that live outside the code: a terminal reachable only from the office LAN, an operator the OS already authenticated, a fixed-width form field that made over-long input physically impossible, a batch job that only ran from a trusted host. Re-hosting that logic behind HTTP removes every one of those silently — the code looks identical, so nothing appears to have changed, and the application is now internet-reachable with no authentication and no field-length enforcement. The `S-5` finding in the sample report is exactly this shape.

A companion improvement: aggregate `migration-introduced` findings **across projects**. A flaw the generator produces every time is worth fixing once in the generator rather than N times in output.

2.3 Framework-aware analysis

Rule packs key on language. Frameworks determine what is actually safe. Blazor Server auto-encodes; Razor Pages needs antiforgery but a bearer-token API does not; Spring Boot's actuator exposure depends on the management port; Django's middleware order changes what a check means.

Add a framework-detection step and framework-specific rule overlays with their own preconditions. This mostly removes the "not applicable to this framework" class of false positive, which is the one most likely to make an engineer distrust the report.

2.4 Verified post-remediation scoring

The projected score is a projection. Close the loop: `--verify-fixes` re-runs only the findings from a baseline, checks whether each is still present, and reports **measured** before/after — which is what the design plan's "verified-after" score was meant to be.

Combined with the Modernizer's `BuildProbeService` and `AppRunProbeService`, each finding's `verification` text becomes an executable check rather than an instruction.

2.5 Coverage metrics

Report what fraction of the attack surface was examined: entry points found versus entry points read, files scanned versus files in scope, categories assessed versus categories in the profile. A score of 85 over 30% coverage is a very different statement from 85 over 95% coverage, and today the report cannot distinguish them.

3. Structural improvements

3.1 Separate the sensor from the judge, formally

Split the skill into a scanner subagent (cheap model, mechanical, produces candidates) and a triage agent (strong model, reads code, judges). Benefits: the expensive model only sees candidates worth reading, cost drops on large codebases, and the two layers become independently testable.

Risk to manage: the cheap layer must not silently drop candidates the strong layer needed. Its recall, not its precision, is the property to test.

3.2 Parallel triage by category

Findings in different OWASP categories are independent. Fan them out — one agent per category — and merge. Meaningful wall-clock reduction on `deep` runs. Watch for the failure mode: parallel agents each see part of the code, so cross-category findings (an auth gap that is also an SSRF pivot) get missed. Reconcile in a merge step that explicitly looks for them.

3.3 Incremental audits

Audit only what changed since a baseline commit, and carry unchanged findings forward. This is what makes per-commit CI economically sensible. Requires per-finding content hashes so a moved line does not read as a new finding.

3.4 Cross-project trend tracking

A per-project history of scores, findings and cost. Answers questions no single report can: is our security posture improving, which categories recur across teams, which rules never fire (dead rules), and which fire constantly and get suppressed (bad rules).

For the Modernizer specifically, this is how you find out whether the generator is getting safer.

4. Report and usability

Improvement	Why
Evidence permalinks	Link <code>file:line</code> to the repo host (GitHub/ADO/GitLab) at the audited commit. One click from finding to code.
Record the commit SHA	The report is currently dated but not pinned to a revision. A finding without a revision is unreproducible.
Diff view against baseline	New / persisting / resolved as a first-class report section, not just per-finding tags.
Ticket export	Generate GitHub/Jira/ADO issues from selected findings, one per root cause, with the remediation and verification as the body.

Improvement	Why
Per-finding owner assignment	Map code paths to teams via CODEOWNERS and route findings accordingly.
Executive summary page	One page: score, trend, top three risks, cost. Different audience from the engineer working the list.
Accessibility pass	The report is keyboard-navigable and theme-aware, but has not been screen-reader tested. Add ARIA labelling on the filter chips and the bar chart, and verify with a real reader.
Bundle a print-to-PDF helper	Half done. <code>tools/md2pdf.py</code> already renders Markdown to print-quality PDF via headless Chromium, and is what produced the PDF copies of these documents. Wiring a <code>--pdf</code> flag into the skill is now mostly plumbing: point the same renderer at the generated report instead of a Markdown file. Two gotchas it already solves and any reimplementation must too — Chromium's <code>--print-to-pdf=</code> mishandles output paths containing spaces (render to a temp path and move), and headless Chromium hangs indefinitely if it contends with a running browser's profile (always pass a throwaway <code>--user-data-dir</code>).

5. Operational hardening

5.1 Pricing data freshness

`assets/pricing.json` is a snapshot with a stated date. It will drift.

- Add a staleness check: if `pricingVersion` is more than ~90 days old, the report says the rates may be out of date.
- Optionally fetch live pricing when network access is available.
- Support an org override file so enterprise-discount rates are used instead of list rates.

5.2 Cost guardrails

Add `--max-cost <usd>`: estimate from the inventory before starting, and stop to confirm if the projection exceeds the cap. Also worth having a mid-run check that halts a `deep` run on an unexpectedly large tree rather than discovering the cost afterwards.

This is the mechanism behind the design plan's cost-aware confirmation dialog, and it needs the per-run data the skill now collects.

5.3 Attribute cost per phase

Break the measured cost down by audit step (inventory / rule pass / triage / reporting). Then you can see where the money goes and optimize the right thing — most likely triage, which is also where the value is, so the decision needs data rather than assumption.

5.4 Robustness of the cost collector — partially addressed

Done: there are now three collectors (Python, bash+awk, PowerShell) verified to produce identical output, so cost accounting no longer depends on Git Bash being present. A cross-shell path bug was found and fixed in the process — Git Bash writes `/c/Users/...` watermarks that native Python cannot open, and the silent

fallback to a full-file scan inflated reported cost by ~32×. `cost.py` now translates MSYS paths.

Still open:

- **Concurrent sessions.** Transcript auto-discovery picks the most recently modified file. With two Claude Code sessions running, it can pick the wrong one. Pass `--transcript` explicitly, or have the skill capture its own session id at Step 0.
- **Subagent attribution** uses file mtime against the watermark. Fine for the common case, but it would over-count if a subagent transcript from an earlier task is touched during the audit.
- **Three implementations is two too many.** Now that a Python resolver exists, the awk and PowerShell collectors are pure fallback. If telemetry ever shows Python is reliably resolvable, retiring them removes ~500 lines and a class of divergence bugs. Keep them until then — the zero-install guarantee is worth more than the tidiness.

5.5 Rule-pack signing

If rule packs are distributed across teams, they become an execution-adjacent supply chain: a malicious pattern could exfiltrate matched content through a crafted rule. Ship a manifest with hashes and verify before loading. Low probability, but the failure mode is bad and the fix is cheap.

6. Analysis coverage gaps worth closing

Beyond the current rule packs:

Gap	Why it matters
Authentication/authorization consistency	The highest-value gap. Enumerate every endpoint and its enforcement, then flag <i>inconsistency</i> — a route protected in one place and not another, a check in the UI absent from the service. This is where real breaches live.
Multi-tenant isolation	If the data model has a tenant column, every query should filter on it. One unfiltered query is a cross-tenant leak. Highly automatable and rarely checked.
Business-logic flaws	Negative quantities, client-supplied prices or totals, skipped state transitions, replayable idempotency keys, race conditions on balance or inventory. Not pattern-matchable; needs the model reading intent — and it is what pentests actually find.
Secrets in git history	The skill scans the working tree. A rotated-but-not-purged secret is still in history. <code>git log -p</code> scanning is a natural addition.
IaC and cloud posture	<code>config-iac.json</code> covers common cases; real depth would need provider-aware policy checks.
Client-side supply chain	Third-party scripts, SRI absence, CDN trust.
API-specific issues	OWASP API Top 10 — BOLA, mass assignment, unbounded resource consumption — overlaps but is not identical to the web Top 10.
AI/LLM components	If audited code calls an LLM: prompt injection, tool-permission scope, untrusted content reaching a tool call. Increasingly common and entirely uncovered today.

7. Recommended sequencing

Phase 1 — trustworthiness (do this before anyone relies on the score) ~~1.1 golden-corpus tests~~ ✓ · 1.2 consistency harness · 1.6 unmissable dependency caveat · 5.4 collector robustness ✓ (partial)

Nothing else matters if the tool's own output is not known to be stable. The corpus tests in particular are what let you change rule packs without fear — and they immediately proved the point by finding an inverted injection rule.

Next in this phase: **1.2, the triage consistency harness**. With the deterministic layer now under test, the remaining unknown is the variance of the judgment layer, and that is what decides whether the score is gate-quality or indicative-only. Extending the corpus to the four uncovered packs is the natural companion task.

Phase 2 — pipeline fit 1.4 SARIF · 1.5 suppressions with expiry · 3.3 incremental audits · 5.2 cost guardrails

This is what turns a report generator into something that lives in CI.

Phase 3 — analysis depth 2.1 taint analysis · 2.3 framework awareness · 6's auth-consistency and multi-tenant checks

Phase 4 — Modernizer-specific 2.2 both-sides-of-migration auditing · 2.4 verified post-remediation scoring · 3.4 cross-project trends

For the Modernizer, 2.2 is arguably worth pulling forward into Phase 1 — it is the difference between a security report about a generated app and a feedback loop that makes the generator itself produce safer code.

8. What not to do

A short list, because each of these is tempting and each makes the tool worse:

- **Do not add rules without false-positive guards.** Precision is the product. A rule that fires on every `execC` teaches engineers to ignore the report, and that loss is permanent.
- **Do not let the skill fix code by default.** An auditor that edits is an auditor whose findings cannot be independently trusted. Keep remediation an explicit, separate, build-verified step.
- **Do not score unassessed categories as passing.** The current `null` handling is the honest behaviour and the temptation to "fill in" those cells for a nicer dashboard should be resisted.
- **Do not tune the weights to produce nicer numbers.** A score that can be tuned per project describes nothing and compares to nothing.
- **Do not let a required dependency creep in.** The zero-install property is why this skill can drop into any project. If depth requires a real AST engine, make it an opt-in enhancement that degrades cleanly — never a prerequisite.
- **Do not report unverified dependencies as clean.** Of everything here, this is the failure most likely to cause real harm, because it converts an absence of information into false assurance.